Trabajo Práctico – Ejercicio Integrador FastAPI: Clientes

Módulo Clientes
El módulo Clientes fue desarrollado como una extensión del sistema base, siguiendo la arquitectura modular propuesta en el trabajo práctico. Su objetivo es gestionar información de clientes mediante una API REST, permitiendo crear, consultar y filtrar datos de manera estructurada.

Sigue el flujo: Request → Router → Service → Respuesta.

Router (entrada de datos → recibe petición y delega la lógica)
El router define los endpoints que puede usar el usuario:

- GET /clientes → lista clientes.
- POST /clientes → crea un cliente.
- GET /clientes/{id} → busca un cliente por ID.

También, permite filtros mediante query params:

- ?nombre=
- ?email=

Service (lógica de negocio)
Contiene la lógica de negocio/sistema: creación, almacenamiento (en memoria), búsqueda y filtrado de clientes. Incluye validaciones como evitar emails duplicados y bloquear el nombre 'admin'. Si una regla no se cumple, devuelve un error.

Schemas (validación)
Validan los datos con modelos Pydantic:

- ClienteCreate → valida datos de entrada (nombre mínimo 3 caracteres y email válido).
- ClienteResponse → define la respuesta.

Si los datos son incorrectos, FastAPI devuelve un error 422 automáticamente.

Funcionalidades

- Crear clientes
- Listar todos los clientes.
- Buscar por ID.
- Filtrar por nombre y/o email.
- Validar datos automáticamente.
- Aplicar reglas de negocio.

Manejo de errores

El sistema contempla distintos tipos de errores lo que asegura que la API sea robusta y controlada:

- 400: error de negocio → reglas de negocio (ej: email duplicado).
- 404: cliente no encontrado.
- 422: error de validación automática (pydantic).
- 201: caso exitoso.

Ejemplo request exitoso (201)

POST http://127.0.0.1:8000/clientes/
Content-Type: application/json

```
{
  "nombre": "Tomas Rodriguez",
  "email": "tom@gmail.com"
}
```

Response:

Status: 201 Created

```
{
  "id": 1,
  "nombre": "Juan Perez",
  "email": "juan@gmail.com"
}
```

Ejemplo error de negocio (400)

POST http://127.0.0.1:8000/clientes/
Content-Type: application/json

```
{
  "nombre": "Pedro",
  "email": "tom@gmail.com"
}
```

Response:

Status: 400 Bad Request

```
{
  "detail": "El email ya está registrado"
}
```

Ejemplo de cliente no encontrado (404)
GET http://127.0.0.1:8000/clientes/999

Response:
Status: 404 Not Found

```
{
  "detail": "Cliente no encontrado"
}
```

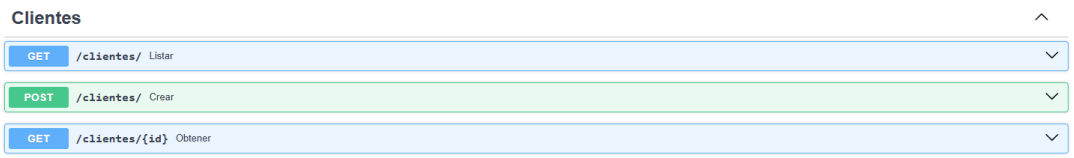
Ejemplo de error de validación (422)
POST http://127.0.0.1:8000/clientes/
Content-Type: application/json

```
{
  "nombre": "Jo",
  "email": "correo_invalido"
}
```

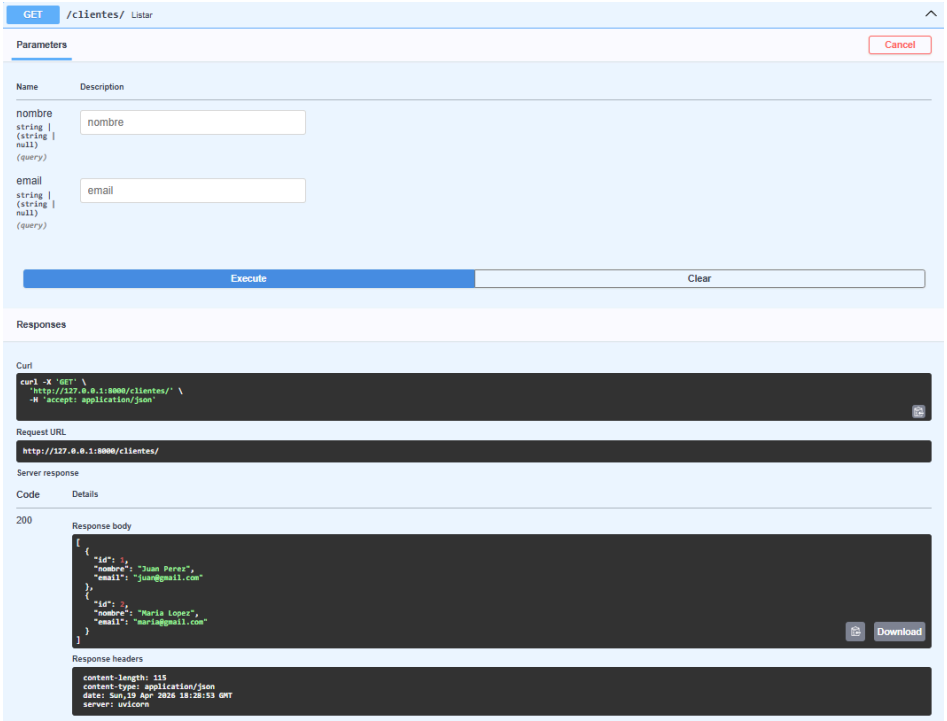
Response:
Status: 422 Unprocessable Entity

```
{
  "detail": [
    {
      "loc": ["body", "nombre"],
      "msg": "ensure this value has at least 3 characters"
    },
    {
      "loc": ["body", "email"],
      "msg": "value is not a valid email address"
    }
  ]
}
```

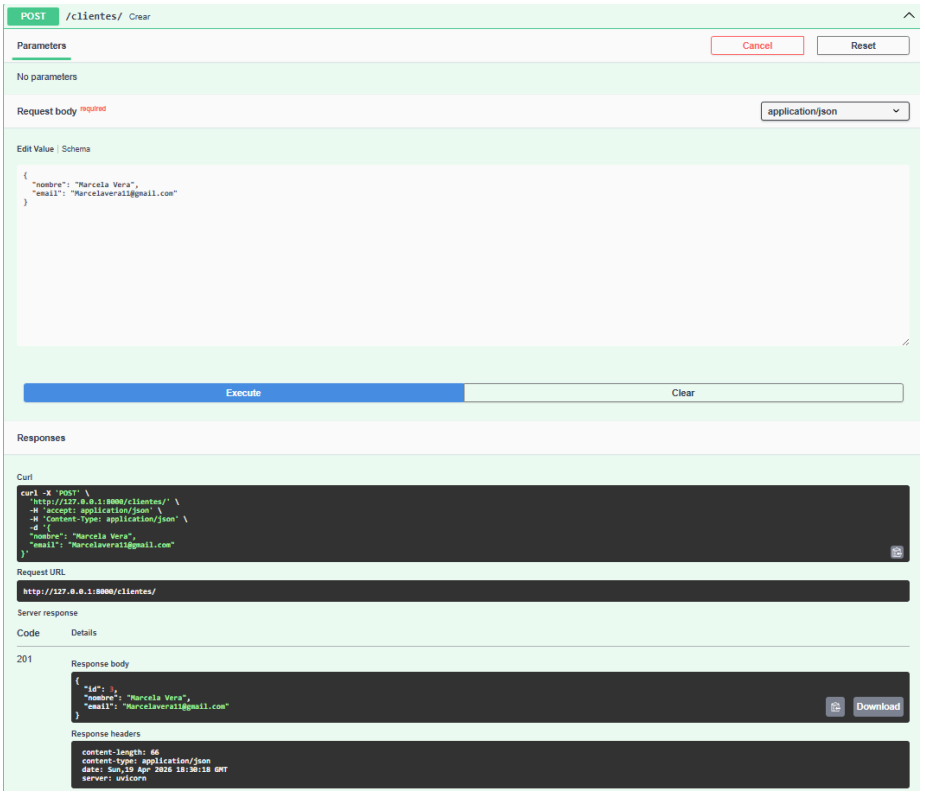
Swagger
Esta captura de pantalla muestra los endpoints disponibles:



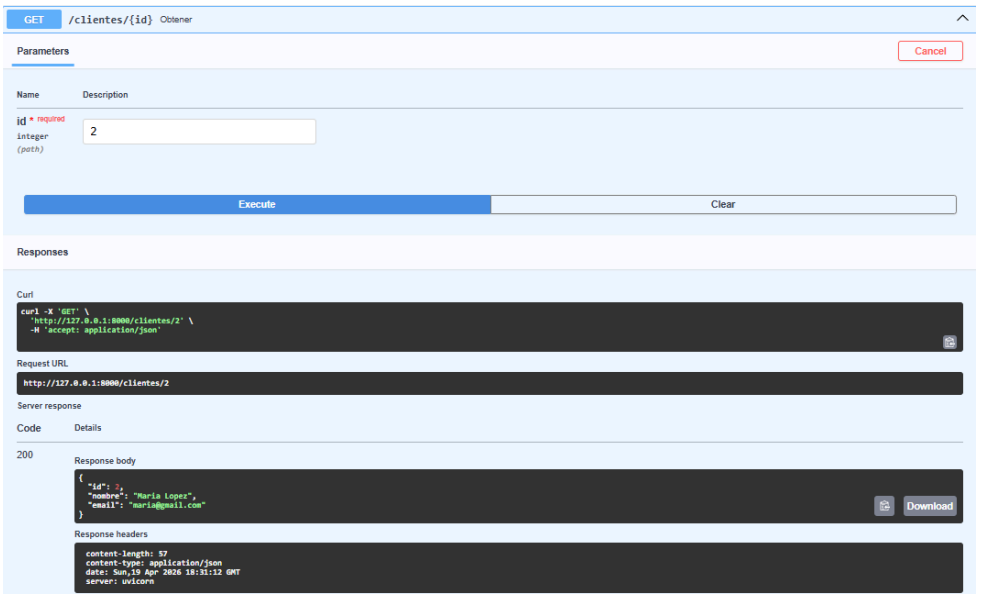
En esta captura de pantalla, se puede ver la ejecución del endpoint de listado con un Status: 200 OK.



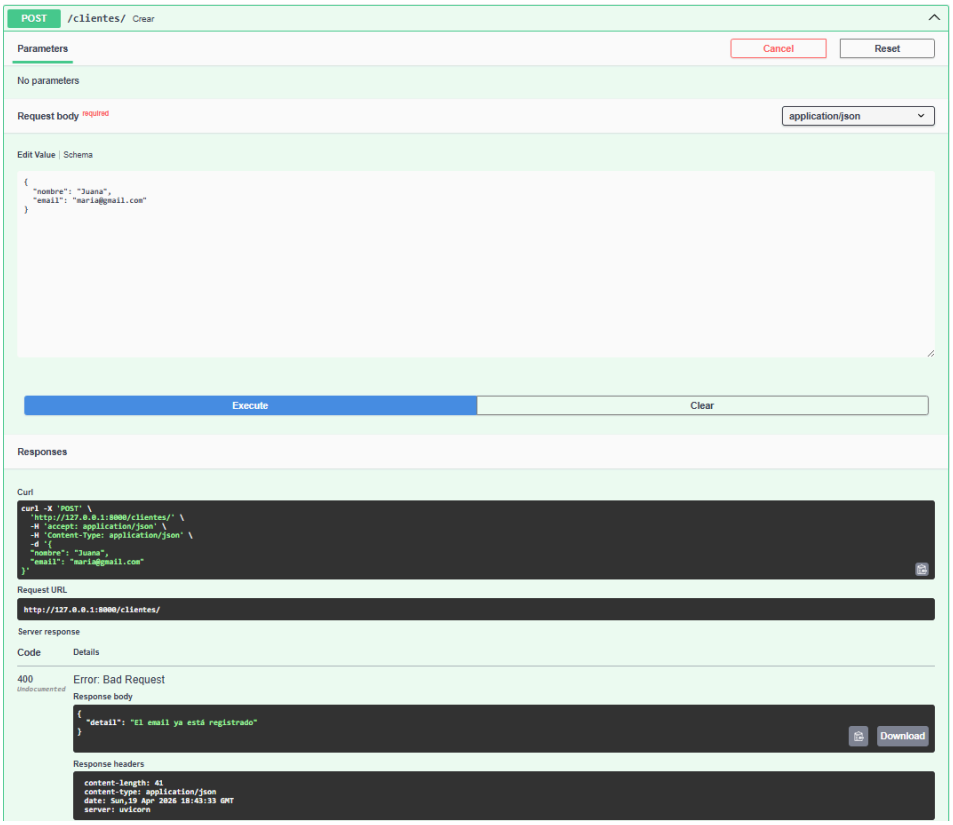
En esta captura de pantalla, se puede ver la ejecución del endpoint de creación de un cliente. El request se realiza con datos JSON. Si el response exitoso, el cliente se crea con un ID.



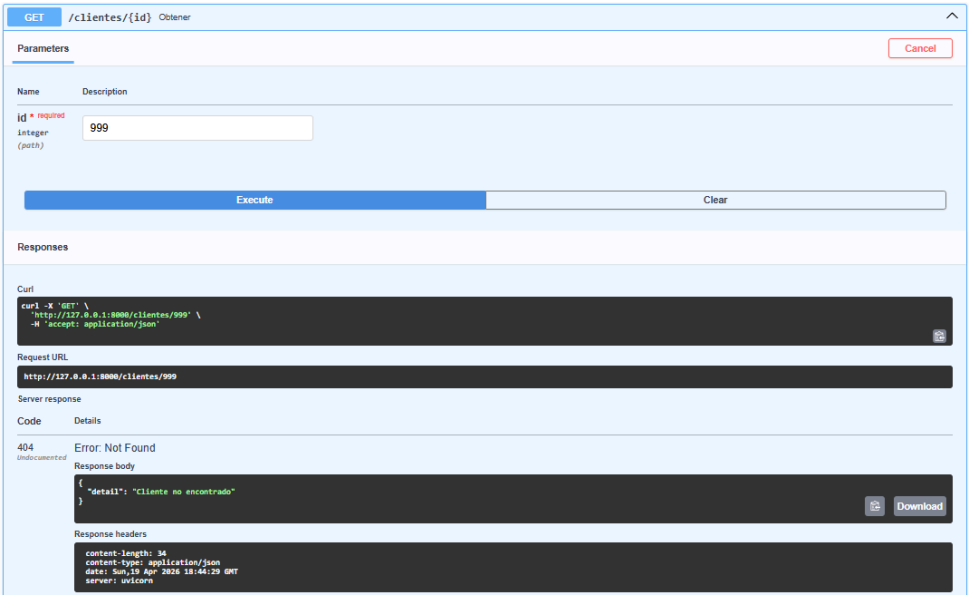
En esta captura de pantalla se puede observar la búsqueda de un cliente por ID. Si la respuesta es Status: 200 OK quiere decir que el cliente fue encontrado correctamente.



En esta captura de pantalla se puede ver la ejecución del endpoint POST/clientes con un email duplicado lo que deriva en un Status: 404 Bad Request. Esto demuestra que las reglas de negocio evitan la duplicación de datos.



En esta captura de pantalla se puede ver la ejecución del endpoint GET/clientes/{id} con un ID inexistente; por eso es que devuelve un Status: 404 Not Found. Esto demuestra un correcto manejo de errores cuando el recurso solicitado no existe.



Link repositorio: https://github.com/denegritomas03/Integrador.git